

Virtualized End-to-End Management Functions for Aggregated Control of Video Traffic Flows

Kaliappa Ravindran, Ancy Cherian, and Arun Adiththan

City Univ. of New York (CUNY - City College)

New York, NY

ravi@cs.cuny.cuny.edu;~ancy.ann@gmail.com;~arunadiththan@gmail.com

ABSTRACT

The paper describes the management and control (M&C) functions of various network nodes in an end-to-end rate-adaptive video transport system. Mobile user devices download video clips by sharing the underlying network path from an ingress node. At the core software level, M&C functions realize the well-known AIMD (additive increase multiplicative decrease) based video rate control algorithm to handle congestion along the path. AIMD is exercised on the aggregated data flows at a source ingress node based on the 'loss reports' signaled from the receiver egress node. Our aggregated AIMD-based control reduces the signaling overhead, relative to the existing approaches that anchor an AIMD instance on each user device itself. This offers scalability, while improving the user-experienced QoS: namely, low jitter in transfer rate, fast convergence to a final rate, and fair sharing of bandwidth. The offloading of aggregated AIMD-based control to the in-network overlay nodes also allows a reduction in the overall bandwidth usage. The software handling of 'last-mile' issues in the path between user devices and egress nodes (such as greedy users and access network channel sharing) are discussed, in a context of fine-granular video encoders in the devices. The paper also shows a virtualization of our M&C functions (as VNF modules) for deployment in large-scale video distribution networks: such as YouTube.

CCS CONCEPTS

• **Computer systems organization** → **Video Distribution Networks**; *Bandwidth Allocation*; *Flow Scheduling*; • **Networks** → *Network Management*;

KEYWORDS

AIMD-based Rate Control, Quality of Adaptation, Scalable Video Encoding, Multi-step Congestion Recovery.

ACM Reference Format:

Kaliappa Ravindran, Ancy Cherian, and Arun Adiththan. 2018. Virtualized End-to-End Management Functions for Aggregated Control of Video Traffic Flows. In *Q2SWinet '18: 14th ACM Symposium on QoS and Security for Wireless and Mobile Networks, Oct. 28–Nov. 2, 2018, Montréal, Québec, Canada*. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3267129.3267142>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

Q2SWinet '18, October 28–November 2, 2018, Montréal, Québec, Canada

© 2018 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-5963-4/18/10.

<https://doi.org/10.1145/3267129.3267142>

1 INTRODUCTION

We consider a rate adaptive end-to-end video transport system (BAVT) realized over a bandwidth-limited network path. The network setting may possibly be a UDP-IP transport connection or a MPLS-based path over IP, which does not allow direct bandwidth reservations¹. Our paper focuses on the management of BAVT from end-user perspective of streaming QoS (quality of service) in a session involving multiple clients. Figure 1 shows the architecture of a BAVT delineating two functions: i) Core adaptation to adjust the video send rates, and ii) System management for external control of session-wide fairness. [1] discusses fairness issues in DASH systems that employ HTTP-over-TCP streaming.

The intrinsic complexity of BAVT arises from two estimation problems: the available bandwidth along the path, and the bandwidth demand of a video data flow exercised on the path. Often, the data flow from a video source is bursty, making its bandwidth demand highly unpredictable. If multiple video sources share the path, the statistical multiplexing effects also impact the bandwidth use. The lack of knowledge about bandwidth use is compounded by a presence of cross-traffic along the path, thereby making the available bandwidth B_{av}^* difficult to estimate [2, 3]. If λ is the rate at which the video source attempts to send packets, we denote the bandwidth demand of the flow as: $F^*(\lambda, [m, \rho, \dots])$ — where m is the residual packet loss rate on the link and ρ depicts the burstiness of packet flow. With an end-point aggregation of video flows over the data path, ρ depends on the number flows admitted N and the traffic smoothing exercised on flows [4]. Since neither B_{av} nor F^* is known in exact form, the adaptation condition: $F^*(\lambda', [m, \rho]) < B_{av}$, is determined by model-based control — where λ' is the sustainable send rate. In this light, [5] suggests an adaptive scheme to estimate the TCP throughput (by sampling the network state: RTT and window size) for use as approximate bandwidth B_{av} to control the bit-rate generated by fine-granular video encoders (such as H.264). The scheme however considers a single video stream — and hence its use with multiple streams over a shared TCP path is not clear.

In the absence of precise knowledge about B_{av}^* and F^* , an AIMD algorithm that employs *additive increase multiplicative decrease* of video send rates is quite effective for the BAVT [6, 7]. With long-term stationary traffic, the AIMD algorithm works well to get around the difficulties posed by the uncertainty in knowledge of network state. The AIMD algorithm however does not work well when anchored on end-user devices (say, smart-phones), due to the non-stationarity of last-mile traffic conditions.

¹Even on a software-defined network (SDN) with the ability to dynamically switch the path to different routes, a congestion scenario may arise due to uncontrolled cross-traffic flows or an inability to find non-congested routes.

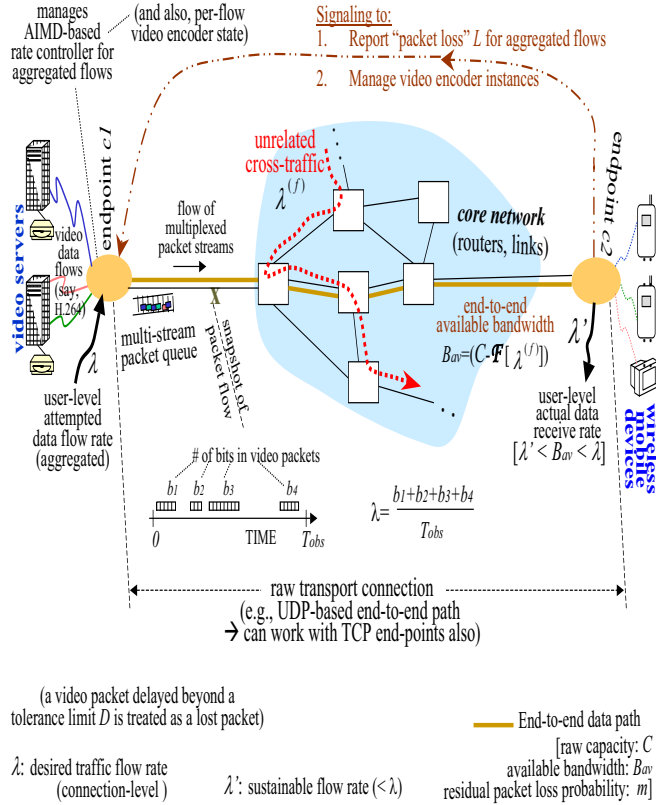


Figure 1: Rate-adaptive video transport system

See Figure 1. We run the AIMD algorithm at the flow aggregation point ($[c_1, c_2]$), and then split an AIMD-computed rate reduction (or increase) among the video receivers sharing this path. In contrast, existing systems have the AIMD algorithm run at end-user devices themselves: which limits the scalability. The AIMD signaling overhead in our approach is reduced by a factor $O(N)$, where N is the number of flows sharing the path. Furthermore, the aggregation point serves to separate the AIMD-based adaptation mechanisms exercised on a shared path between the ingress and egress points at network edges (i.e., c_1 and c_2) from the last-mile issues arising in access network segments that connect the client devices: such as channel fading, channel failures, and faulty devices. The architectural separation allows installing protection mechanisms at network edges to ameliorate the effects of last-mile issues on the video delivery path².

The paper provides an architecture of BAVT system to incorporate the aggregated video traffic handling. The architecture incorporates *virtualized network functions* (NFV) to handle the bandwidth congestion that arises in shared paths connecting the flow aggregation point to individual receivers (over the core network

²The AIMD algorithm is oblivious to the flow aggregation process. So, a termination of AIMD signaling at the aggregation point $[c_1, c_2]$ allows separately dealing with the last-mile issues pertaining to the individual flows and their cross-flow effects.

and/or access network). A focus is on the aggressiveness of one or more flows in adjusting their send rates without conforming to the AIMD-allocated rate splits. A deviation from the AIMD-compliant rates may be due to either a lack of global knowledge of the network state or a client-level uncooperative behavior. The end-point signaling structure to deal with such issues is also described.

The paper is organized as follows. Section 2 shows a black-box view of AIMD-style rate controlled transport path. Section 3 describes the signaling interactions between the AIMD-aggregation point and end-user devices over 'last-mile' path segments. Section 4 gives a system-level architecture of video distribution networks to virtualize the AIMD-based control functions in the form of NFV. Section 5 concludes the paper.

2 BLACK-BOX VIEW OF RATE ADAPTATION MODULE

During congestion, a model-based description of the raw transport sub-system embodied in a BAVT (bandwidth-adaptive video transport) system is given as:

$$L^* = \left(1 - \frac{B_{av}^*}{F^*(\lambda, [m, \rho, \dots])}\right) \quad (1)$$

where $B_{av}^* < F^*(\lambda, [m, \rho, \dots])$ and L^* is the packet loss rate experienced ($L^* > m$). In the model, B_{av}^* constitutes the internal state of end-to-end transport connection that forms the core of BAVT system and L^* depicts the system output resulting from an injection of a data flow λ over the connection. The observed packet loss L can be used by the end-points $[c_1, c_2]$ to lower the video send rate to a level that matches the available bandwidth B_{av}^* . We have employed this model-based view in an earlier work, for the stability analysis of video rate control process [8]. In contrast, our present work focuses on virtualization of the AIMD rate control process, and its incorporation as NFV in a system-level architecture.

2.1 AIMD-based rate adaptation

In the absence of exact knowledge about $F^*(\dots)$ and/or B_{av}^* , an *additive increase multiplicative decrease* (AIMD) algorithm [6, 7] is employed as a coarse approximation to adjust the send rate λ' matching the available bandwidth B_{av}^* .

Whether a rate reduction relieves the congestion or not can be ascertained only by exercising the rate reduction on the transport connection and observing the end-to-end packet loss rate L therein. If $L > \delta_h$, the under-reduction in send rate is compensated in a subsequent rate reduction. The observe-and-adapt cycle continues over multiple iterations q until the congestion is relieved, i.e., $L < \delta_l$. The total time to adapt: $T_c \approx q\tau|_{q \geq 1}$, depicts the convergence latency of rate adaptation process (i.e., *control round*).

The adaptation module programmatically manages the rate convergence process by abstracting the transport connection as a computation function of the form: $L = \text{net}(\lambda)$, where $\lambda > 0$ and $L > 0$. The internal details of $\text{net}(\dots)$ to determine L for an input λ are not known to the programmer of network control, i.e., $\text{net}(\lambda)$ appears as a black-box taking λ as the input and returning L as the output. Due to monotonicity condition, the programmer however knows how the function $\text{net}(\dots)$ behaves when λ changes:

$$\text{net}(\lambda + \zeta) > \text{net}(\lambda) > \text{net}(\lambda - \zeta) \quad (2)$$

for $\zeta > 0$. In other words, an increase in λ leads to a higher L , and a decrease in λ leads to a lower L .

Suppose λ_0 is an initial input for which $\text{net}(\lambda_0)$ returns a value $L_0 > \delta_h$. An algorithmic procedure reduces λ in multiple steps of $(\beta \times L)$ decrements to a value λ_f such that $\text{net}(\lambda_f) < \delta_l$. When $L < \delta_l$, the procedure increases λ in multiple steps of a constant increment α until reaching a rate $\lambda_{f'}$ such that $\text{net}(\lambda_{f'}) > \delta_h$. Thereupon, the procedure switches back to the rate reduction cycle. The sustainable send rate is given by: $\lambda' = \frac{\lambda_f + \lambda_{f'}}{2}$, where $0 < \lambda' < \lambda$.

The repetitive computational steps that realize the AIMD procedure (i.e., control iterations) look like the following:

```

L := net(λ) /* Initial output L > δh */
λ(g) := λ;
state := CONG;
while (true)
  if (state = CONG) /* path is congested */
    lth := δl;
  else
    lth := δh; /* Recall that 0 < δl < δh */
  if (L ≥ lth) /* loop 1: multiplicative decrease */
    λ(g) := λ(g) - βL;
    state := CONG;
  else /* loop 2: additive increase */
    λ(g) := λ(g) + α;
    state := ¬ CONG;
  L := net(λ(g));
  if (steady_state is reached)
    break;
λ' := λ(g).

```

Each iteration of "while" loop 1 realizes the multiplicative decrease phase of AIMD rule, whereas "while" loop 2 realizes the additive increase phase — where τ_{conn} is per-iteration time-interval. Given the monotonicity property of $\text{net}(\dots)$ function, the AIMD procedure is guaranteed to terminate, i.e., find λ' in a finite number of iterations.

The AIMD procedure is folded into a *virtualized network function* (NFV), i.e., a programmable black-box, that takes $[\alpha, \beta, (\delta_l, \delta_h)]$ as inputs from the management-plane and $[L, \lambda]$ as inputs from the control-plane, generating λ' as the resulting output. The procedural hook to observe L from the environment is a part of the NFV specification. The AIMD procedure gets invoked upon the NFV receiving signal from an observer module to notify congestion: i.e., when the condition $L > \delta_h$ occurs. This may occur due to a depletion in the available bandwidth of network B_{av}^* or an increase in the send rate of video sources λ . The inability to disambiguate these situations, combined with the propensity to search for a higher send rate when $L < \delta_l$, leads to a continuous execution of the AIMD procedure even in the steady-state. In this light, existing video adaptation algorithms are either too sluggish to respond to the changes in congestion levels or too sensitive to short-term network bandwidth variations (both affect the user QoS) [9].

The flow managed by AIMD is itself an aggregation of N video flows whose packets are multiplexed over the transport path ($N \gg 1$). The aggregation reduces the amount of state kept at end-points and the associated signaling overhead in core network, by $O(N)$.

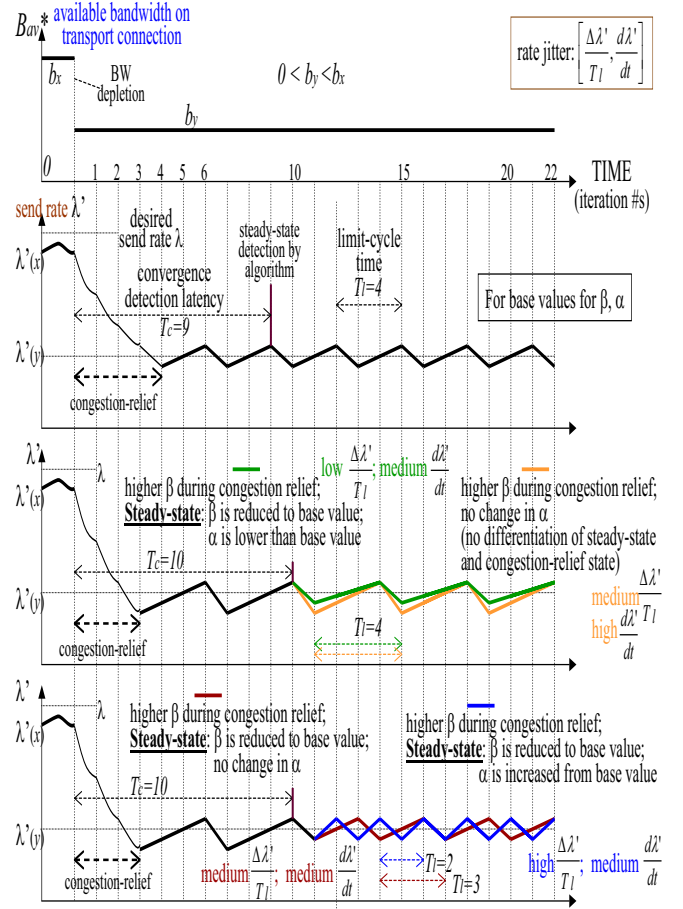


Figure 2: AIMD Rate sharing among multiple sources

2.2 Moderated rate increase/decrease cycles

Our extension to AIMD allows a *rate moderation* when computing the reduction or increase of send rate, as the case may be, in response to a packet loss observation L . It allows a rapid rate adjustments in the congestion-relieving state, while slowing down the rate adjustments in the steady-state. The moderated rate adjustments purport to lower the convergence detection latency $T_{c, \text{AIMD}}$ when relieving a congestion and to increase the rate limit-cycle period $T_{l, \text{AIMD}}$ in the steady-state (i.e., lower the rate jitter). Figure 2 shows a sample timing scenario to illustrate our software-controlled moderation of rate adjustments. For comparison, the behavior arising from a static setting of the α and β parameters, as in existing works, is also shown. See [8, 10] for the algorithmic details of our extended AIMD.

With moderated rate adjustments in the steady-state, the rate jitter can be reduced without increasing the time taken to relieve congestion. As shown in Figure 2, a dynamically managed adjustment of the send rate in the congestion-relieving and post-relief states purports to lower the rate jitter relative to a base setting

of AIMD parameters (β, α) . When no differentiation is made between the congestion-relief and steady-state phases, a parameter setting to lower rate convergence latency incurs the side-effect of increasing the rate jitter. Likewise, an increase in α increases the rate jitter, but offers the benefit of lowering the convergence detection latency. The QoS performance of BAVT system can be improved simply by using a higher β during congestion-relief, and then falling back to the base setting when steady-state is reached. This parametric control is enabled by virtualized AIMD functions.

The rate convergence process as part of a congestion relief and recovery is managed at the level of N aggregated video flows. With a large number of flows aggregated (i.e., $N \gg 1$), the relief & recovery trauma is amortized among these flows to lessen its negative effects on the end-users.

2.3 Handling of aggregated video flows

For rate adaptation purposes, an aggregation of the N flows is treated as a composite flow:

$$\lambda = \lambda_1 \oplus \lambda_2 \oplus \dots \oplus \lambda_N;$$

where ' $\oplus$ ' is the aggregation operator. Here, a rate adaptation $\Delta\lambda$ as computed by AIMD for the aggregated flow unfolds as a set of increase/decrease actions $\{\Delta\lambda_i\}_{i=1,2,\dots,N}$ exercised on the N component flows such that $\Delta\lambda = \oplus_{i=1}^N \Delta\lambda_i$. The flow aggregation operator ' $\oplus$ ' exhibits *weak additivity*: namely, a perceived bandwidth gain accrued by statistical multiplexing of the N data flows under some (limited) knowledge about their traffic behaviors (say, the peak and short-term average rates). The multiplexing gain is measured relative to a case of treating each of the N flows separately during a transport multiplexing.

At the architecture level, we provide for a controlled splitting of the AIMD-computed send rate among the N end-user devices that are serviced by the aggregation point ($N \gg 1$). A flow V_i gets a rate allocation share:

$$\lambda'_i = \frac{\lambda' \times w_i}{\sum_{n=1}^N w_n},$$

where w_n is the normalized weight assigned to a flow V_n , say, on a scale 0-1, relative to the other flows (a fair-share is depicted by equal weights). The traumatic effects of congestion relief & recovery are thus amortized across N flows, thereby lowering the negative impact on per-user QoS.

For an aggregated treatment of N video flows, the end-points c_1 - c_2 interact with two types of entities: i) the end-user devices receiving video data; and ii) the video content hosting sites (i.e., sources). Refer to Figure 1. The interactions (i) and (ii) are orchestrated via signaling channels realized over the access networks. The source sites may be geographically spread out over a network of VMs and disks (say, a data-center) that host the video data to be played out. The connections from these sources to the end-point c_1 are typically realized over high capacity network paths – and hence are less vulnerable to congestion. In contrast, the connections from receiver devices to the end-point c_2 are typically low-to-medium capacity shared network paths (often, wireless links) that are subject to wide fluctuations in bandwidth. Thus, an adaptive allocation of video download rates to the receiver devices factors

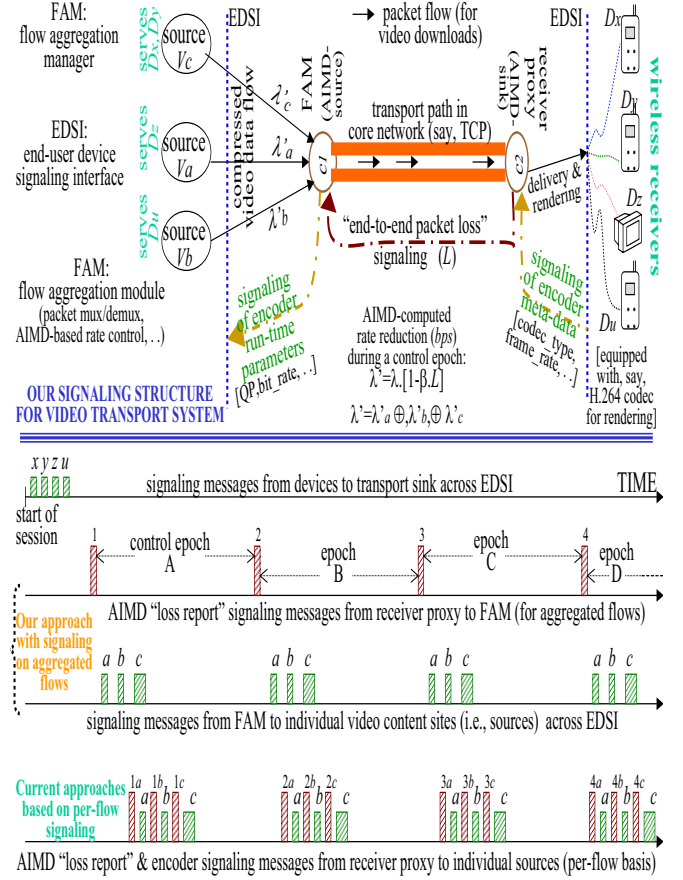


Figure 3: Signaling structure for aggregated control of video flows

in both the bandwidth congestion in core transport network (i.e., $F^*(\lambda, \dots) > B_{av}^*$) and the last-mile issues in local access network.

Besides a reduction in the signaling overhead, our approach of video flow aggregation offers a higher potential to reap the statistical multiplexing gains in bandwidth use by consolidating the traffic behaviors of component flows.

2.4 Structure of BAVT signaling system

Figure 3 shows the underlying signaling structure of BAVT system to support an aggregated treatment of the video flows. The signaling from end-point c_1 to the video source sites occurs at every AIMD control cycle: namely, loss-report L from c_2 and rate adjustment by c_1 therein. The signaling, which occurs over the access network at c_1 , purports to supply the allocated bit-rates for each of the N video encoder instances running at K source sites – where $1 \leq K \ll N$. Note that neither the core network path between c_1 - c_2 nor the last-mile path between c_2 and receiver devices carry these signaling messages. This is unlike the existing BAVT systems where a signaling for the AIMD and source-level control occurs all the way between receiver devices and source sites.

For a BAVT system with 3 sources and 4 receivers (i.e., $K = 3$, $N = 4$) shown in Figure 3, our method incurs a signaling overhead of 4 AIMD-level messages and 12 end-point control messages over a 4-cycle period. In contrast, existing per-flow signaling methods would incur 12 AIMD-level messages and 12 end-point control messages, traversing all the way between receivers and sources.

The per-flow signaling actions triggered by c_2 on the K source instances at c_1 (one per device) are synchronized with the loss-reports from c_2 , because each loss-report event triggers a potential adjustment of the video send rate of each of the N flows. We thus piggyback the N meta-data elements needed for per-flow rate control on the AIMD signaling messages (say, carried over RTCP-based control channels). The signaling of encoder actions on the K source sites, as triggered from c_1 , incurs an overhead $O(K)$ — which is much lower than the $O(N)$ overhead in existing approaches. Note that a signaling of the receiver-device configuration meta-data to c_2 (such as encoder type and screen resolution & size) occurs only at the start of video transport session.

As can be seen, the benefit of $O(N)$ reduction of signaling message overhead on the core transport path to handle AIMD loss-reports accrues on the access network paths at source and receiver ends as well.

We now describe the specifics of end-point signaling for the control of video encoders to generate adjustable bit rates in response to AIMD-triggered rate allocations.

3 SIGNALING OF VIDEO SOURCES

AIMD-computed rate adjustment in a control iteration is split among the end-user devices that are serviced by the flow aggregation point: say, c_1 . The exact rate split is determined by a session-layer strategy such as source-priority based and weighted-fair allocation. Flow-specific rate allocations determined therefrom are then signaled by $[c_1, c_2]$ to the various end-user devices $\{x(c_1)\}$. Figure 4 shows the system architecture that connects the AIMD aggregation point to the end-user devices. Without compromising generality, we assume a H.264 encoder in each device that generates a fine-granular scalable video data to achieve low bit rates.

3.1 Intra-source control of encoder bit-rate

Given a bit rate allocation λ_x for a device x , the H.264 encoder housed in x does not have a direct way of mapping λ_x onto the scene quality. Instead, x may possibly realize a control algorithm that progressively adjusts the encoder parameters based on the observed bit rate from x . In the absence of such a closed-loop algorithm, the encoder generates a bit rate that is based on statically set encoder parameters (i.e., open-loop control). In either case, the encoder implements basic techniques that remove redundant information from the bit-stream (e.g., motion compensation) — which lowers the bit rate generated.

Video compression involves a quantization mechanism as well that allows the grouping of spatial details in the scene captured — *albeit* with increased distortion and loss of perceptual quality [11]. How much spatial detail is retained is determined by a quantization parameter (QP), where a higher QP depicts the removal of more details — and hence a lower bit-rate. As the scene complexity increases, maintaining the same bit rate requires a higher

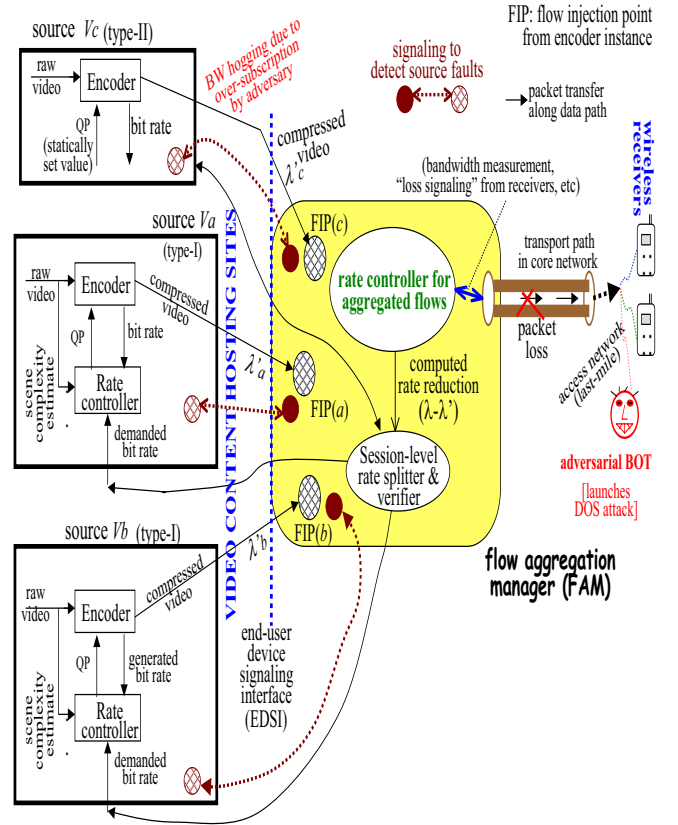


Figure 4: Architecture for video encoder control

QP: which in turn lowers the perceptual QoS at end-user level. An empirical view of QP versus bit rate versus scene complexity is also shown in Figure 4. Additionally, a device-internal algorithm optionally realizes a rate control functionality that adjusts the encoder parameters (such as QP) to achieve a target bit rate [12]. The encoder-internal rate control mechanism³ involves providing an estimate of scene complexity to the control logic that in turn adjusts the QP.

We have conducted experimental studies on video bit rates generated by H.264 encoders, using the FFMPEG software (available in the public domain). A sample run involving about 900 video frames played at 25 *fps* (i.e., for 36 *sec*) was analyzed. Figure 5 shows the bit-rates generated as 1-*sec* time-scale averages: which shows the extent of burstiness in the data flow injected into the transport network. The changes in data generation rates over the observation period arise due to both the variations in scene content and the end-point signaled changes in QP. The FFMPEG also implements a leaky-bucket based traffic smoother, which outputs an effective bit-rate at approximately 250 *kbps*.

³The H.264 Standard does not prescribe a rate control functionality. It only offers some guidance for a possible implementation of rate control.

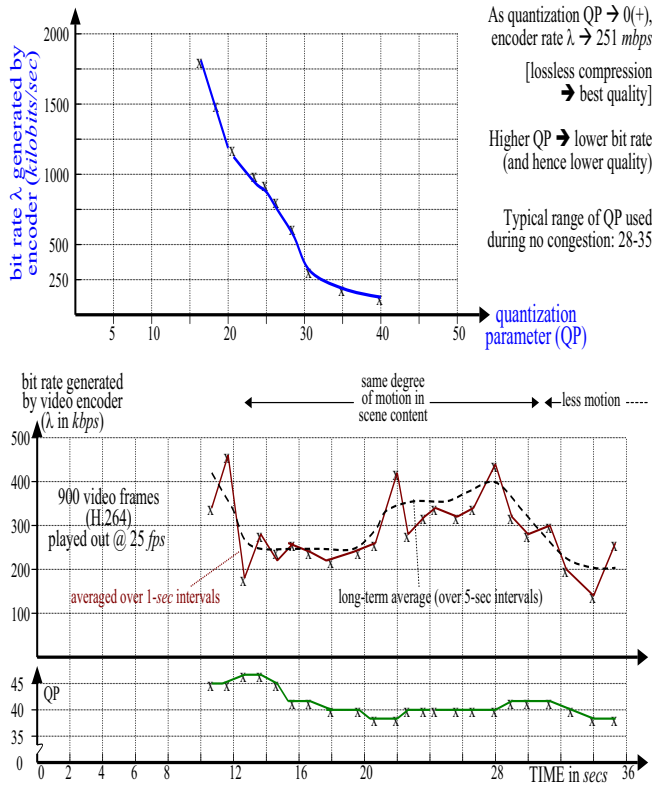


Figure 5: Experimental results on encoder bit-rates

3.2 Heterogeneity of video encoders

Often, an adaptive encoder device encompasses a control logic that employs scene modeling to determine the QP adjustments. We categorize such a device as type-I.

Since the mapping of QP to a bit rate is not exactly known to the encoder, the control logic executes a feedback loop to continuously adjust the QP until the observed bit rate is kept at a desired level. This intra-encoder closed-loop control in source devices (such as V_a and V_b in Figure 4) purports to achieve a CBR output that closely matches a demanded bit rate. The bit rate so generated may however not meet the QoS needs: because the perceptual scene quality achieved, i.e., QoS, depends on the scene complexity. For a given bit rate generated, a higher scene complexity means a higher QP — and hence a lower QoS. In such a scenario, an external (invisible) feedback loop involving the human user may attempt to exercise QoS control by forcing a higher bit rate emanating from the encoder (say, through a GUI). This user-level control involves providing an extra allowance in the demanded bit rate that in turn manifests as lowering the QP.

Consider a case where the encoder in a source does not incorporate the rate control functionality (which we denote as type-II) — such as V_c in Figure 4. At start, QP is statically set to an initial value based on a preprogrammed information on the (average) scene complexity. This open-loop encoding leads to a VBR output

that is possibly quite different from the rate demand on the encoder. Though a desired input rate for encoder is signaled from the session-level rate splitter in the flow aggregation manager (FAM), the signaled rate triggers a static mapping onto a QP value under certain assumed scene complexity. With no internal mechanism to dynamically adjust QP, a deviation in the scene characteristics manifests as generating a different bit rate.

As can be seen, regardless of the type of encoder employed, a significant difference in the bit rate generated relative to a demanded rate does not imply device-level faults.

3.3 Detect non-compliance to allocated rates

The multiple control loops at work in an end-user device x manifest as generating a bit rate higher than the AIMD-allocated rate for x . Such an over-subscribed bit rate emanating from x may however not be due to a malicious intent. Nevertheless, a scenario arises where an end-user device such as x exceeds its allocated rate, whereas another device stays below or at its allocated rate. The disparity of actual bit rates emanating from various end-user devices relative to their allocated rates arises due to the absence of exact knowledge at the AIMD aggregation point about the impact of a rate allocation schedule on the session-wide QoS.

It is hard to detect the non-compliance to the stipulated rate by a device. Nevertheless, the rate allocator in the FAM module may still choose to monitor the bit rate generated by a device. The rate monitoring serves two purposes:

- (1) Notify a type-I device about its send rate, so that it can re-adjust the encoder parameters (if necessary) to scale up/down the send rate⁴;
- (2) Run a supplementary analysis (say, using machine-intelligence tools) to determine if device-level rate non-compliance (if any) is due to a malicious failure.

The FAM module can reschedule the rate allocations, if needed, based on the device capabilities and priorities. FAM maintains a configuration table that lists whether a device is of type-I or type-II, the meta-data about encoder settings, and the user-perceived utility of a send rate.

The last-mile signaling activities between the FAM and N end-user devices for individual rate allocations in our aggregated-AIMD method are much lighter than the end-to-end loss report (ELR) signaling that otherwise occurs on the devices themselves in the existing methods of per-device AIMD. Furthermore, the separation of AIMD control loop from the device-FAM interactions allows the latter to be configured independently and changed dynamically. Besides this architectural flexibility, the FAM provides a secure anchor point to monitor the rate compliance of devices and therein isolate the devices suspected to be faulty. Whereas, the per-device AIMD approach requires that the monitor be implemented in the device itself — which makes it much harder to verify the rate compliance of devices.

3.4 Rate control in the last-mile

Congestion may still occur in the last-mile path segments connecting to end-users, due to external factors such as channel fading,

⁴The feedback loop between the FAM and device encoder executes on top of the encoder-internal control loop that realizes QP-based rate adjustment.

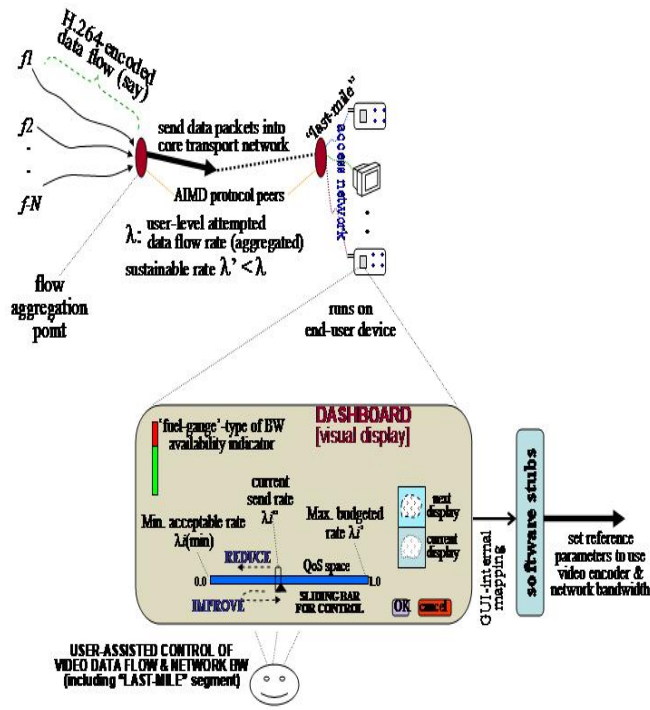


Figure 6: QoS control Dashboard GUI in devices

multi-user interference, and user-device CPU and memory limitations. Given the large plethora of mobile heterogeneous user devices today, the rapidly fluctuating nature of last-mile traffic precludes the anchoring of AIMD at end-points (unlike in the past). Last-mile congestion is hence outside the realm of our aggregated AIMD, and should instead be handled by methods taking cognizance of non-stationary traffic behavior. We advocate an *user-assisted approach* to control the video flow rate over a feasible range, as determined by per-user rate allocations by the FAM. A dashboard-like GUI can be employed, wherein the user can set a flow rate matching the access path characteristics leading to the aggregation point. Figure 6 illustrates the system architecture for last-mile control.

A flow V_i may not be able to fully utilize its allocated send rate λ'_i due to last-mile issues. Any such unused portion of send rate by a source can be used by other sources in fair-share manner. If λ''_i is the actually utilized send rate by the flow V_i , the additional share assigned to a flow V_j from the unused portion of send rate by V_i may thus be given by:

$$\Delta\lambda'_j(i) = \frac{\lambda'_i - \lambda''_i}{N-1},$$

where $\lambda_{i(min)} \leq \lambda''_i \leq \lambda'_i$. Note however that V_j may itself be under-utilizing its allocated rate: $\lambda'_j + \sum_{i=1, i \neq j}^N \Delta\lambda'_j(i)$. In the presence of such uneven last-mile conditions, a max-min fairness criteria can be employed by the FAM to split the AIMD-computed rate

λ' among various flows. As can be seen, there are multiple control loops in a BAVT system: i) AIMD-level, ii) FAM-to-device level, iii) device encoder level, and iv) user-level. These loops interact in subtle ways that impact the system performance and fault-resilience.

4 NFV FOR AIMD & VIDEO ENCODER

We consider the setting of a Content Distribution Network (CDN) that delivers stored video content to multiple client devices in response to their asynchronously triggered pull actions — such as the YouTube. Video chunks are stored in one or more master servers, which then cache the content at intermediate proxy nodes geographically closer to client locations in the distribution topology (i.e., mirror sites). The logical name of a video content generated by client requests is resolved to the nearest mirror site for pull actions on the content data. Here, our NFV plays a role in the content extraction, data transport, and device-level rendering.

4.1 Overlay anchors for video rate control NFV

The CDN creates a tree-structured overlay topology $T(V', E')$ on top of an infrastructure network $G(V, E)$ that is made up of core transport path segments $\{lnk(x, y)\}$ — where the in-network nodes $x, y \in V$ and transport link $lnk(x, y) \in E$. Each link $lnk(x, y)$ may be a TCP or UDP-based transport connection, with an on-tree overlay path consisting of a sequence of one or more transport links. The overlay nodes V' and path segments E' host the functions needed for video content delivery to clients: namely, storage, transport, and rendering — where $V' \subset V$ and $E' \subset E$. One or more on-tree nodes $V'' \subset V'$ function as mirror sites to store the video content data from a master server R for the orchestration of video delivery to various geo-distributed clients over shared core and access networks.

A mirror site $x \in V''$ orchestrates the video delivery to multiple clients C in its geographic vicinity, with the sub-tree rooted at x providing the content data routing path to reach one or more leaf nodes $\{z\}$ for onward delivery & rendering at the client devices. Here, x is the ingress node for a video data flow, with a leaf node z serving as the egress point to deliver the video data at a set of client devices $c(z) \subseteq C$. The overlay path $x-z$ consists of one or more transport segments $[lnk(x, y), lnk(y, \dots), \dots, lnk(\dots, z)]$ that gets mapped onto from a client request: say, carried in a HTTP message. NFV realization of our extended AIMD algorithm for aggregated video flows runs on the nodes x and z , with the AIMD-estimated flow rate proportionately split across $c(z)$.

Figure 7 shows a 19-node topology to distributed video content to seven geo-distributed client clusters⁵. The overlay tree consists of nine nodes, with two of them (nodes 7 and 3) functioning as the content mirror sites. Multiple video encoder instances run at the mirror sites 7 and 3, with the corresponding decoder instances running on the client devices. This allows the management of bandwidth needed for the download actions of clients served by the egress nodes 4,8,9,12. The path segments $lnk(0, 7)$ and $lnk(0, 3)$ transport the content chunks pushed from the master server R for storing at nodes 7 and 3 respectively.

⁵Each client cluster is a logical entity, representing 1000's of client devices. An additive superposition of the traffic flows generated by these devices allows treating them as a single client generating the combined traffic flow.

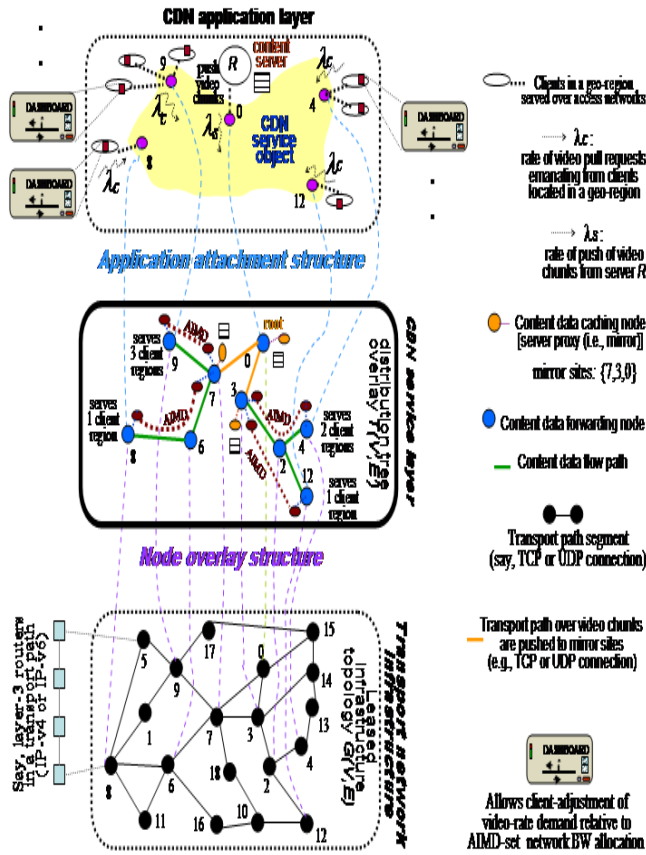


Figure 7: Layered CDN system with BAVT support

The clients $c(z)$ run a dashboard-GUI that gives a handle for users to manage their bandwidth demands relative to the AIMD-allocated per-flow rates — c.f. section 3.4. Bandwidth management is based on the flow rate specs of video chunks, the sharing of download path segments among multiple client pull actions, and the frequency & chunk size of server push actions.

4.2 Management of push & pull of video data

Content chunks at an ingress node are available in different bit rates (by suitable encoding). During play-back, a client attached to the egress node selects from the alternatives the next chunk to download that maximizes the video quality for a given AIMD-allocated bit-rate. Client-level signaling to the ingress node enables a selection of video chunks for transport to the egress node followed by a play-out to the device. The bandwidth needs to transport the video chunks pushed to a mirror site are determined by the chunk push rate η_s and chunk size d_s . Whereas, the bandwidth needs to download selected chunks from a mirror site are determined by the chunk pull rate η_c and d_s . An AIMD-based reduction in the download bit-rates in the event of congestion along the downstream paths manifests as sending a smaller chunk of size

$d'_s \ll d_s$. The bandwidths expended for regular downloads and rate-adjusted downloads of video content and for video content push are determined from the corresponding bit-rates, namely:

$$\lambda_c = \eta_c \times d_s; \lambda'_c = \eta_c \times d'_s; \lambda_s = \eta_s \times d_s;$$

respectively⁶. When the content push and pull occur over disjoint paths, their bandwidth needs can be managed independently. If one or more path segments involved in a content push and pull overlap on the topology, their bandwidth needs are managed by jointly considering $[\lambda_c, \lambda_s]$. Regardless, a client-experienced pull latency is impacted by the bandwidth availability along push path segments and the chunk push rate λ_s . A larger pull latency can manifest as a stall in the video play out — and hence a lower QoS.

Refer to Figure 7. The AIMD algorithm is instantiated on four overlay paths: $[lnk(3, 9)]$, $[lnk(7, 6), lnk(6, 8)]$, $[lnk(3, 2), lnk(2, 4)]$, and $[lnk(3, 2), lnk(2, 12)]$. The latter two overlay paths have a common link $lnk(3, 2)$ — and hence their end-to-end properties (such as packet delay/loss and security) exhibit a correlation. For instance, both these paths will experience an end-to-end congestion if the link $lnk(3, 2)$ reaches a state of bandwidth bottleneck. Likewise, an AIMD-enforced rate reduction on λ_c along the path $[lnk(3, 2), lnk(2, 4)]$ can possibly relieve some congestion on the path segment $lnk(2, 12)$ — and vice versa. So, it is desirable to coordinate the two AIMD instances that control the traffic flows on these paths. On the other hand, the AIMD instances for overlay paths $[lnk(3, 9)]$ and $[lnk(7, 6), lnk(6, 8)]$ do not interact with each other, due to path disjointness — and hence can run in parallel to adjust λ_c . Furthermore, given the disjointness of $lnk(0, 7)$ and $lnk(0, 3)$ from the download path segments, the AIMD-level management of bandwidth for client pull actions from nodes 3 and 7 can be treated independent of server push actions from node 0.

After a tree overlay $T(V', E')$ is set up for a CDN, we install AIMD anchor points at selected on-tree nodes $V'' \subset V'$ to effect on-demand rate adaptive video delivery to clients. Algorithms to generate overlay tree topologies are part of a CDN system to deliver content to clients anyway — and hence is beyond the realm of AIMD-based rate adaptation.

4.3 Signaling overhead for bandwidth control

The overhead for per-device signaling to an ingress node to post the meta-data λ_c needed for on-tree resource management is $O(k.N)$ — where $0.0 < k \ll 1.0$. The small multiplier k factors in the much slower time-scale of per-device signaling for video encoder control. Recall that ELR signaling in our aggregated AIMD approach occurs once for the group of N client devices during a control epoch to determine an adjusted rate allocation λ'_c for the N -aggregated flow — where $\lambda'_c < \lambda_c$. In contrast, existing approaches anchor AIMD-based control right on the client devices and R (i.e., a separate AIMD-instance for each client-server pair) — such as the DASH

⁶A repeated push of video content chunk p from the master server R to a mirror site x may occur due to popularity-driven caching methods: for optimal use of limited storage and/or reduced search overhead (by storage de-cluttering of unused content data). The chunk p cached at x may get removed if: i) lease-time of p assigned by R expires; ii) download requests on p are quite sporadic; or, iii) updates of p have occurred at R that make the cached version of p out-of-date. In this light, η_s pertains to how often p gets reinstalled in x 's cache (say, by a signaling from x to R).

system [13]. The hand-in-hand device-level signaling of ELR feedback for AIMD-control and encoder-level signaling for per-flow bit-rate control (at disparate time-scales) incurs a complexity of $O(N) + O(k.N)$ — which amounts to $O(N)$ for $0.0 < k < 1.0$. This makes the existing approaches less scalable.

Besides the $O(N)$ reduction in signaling overhead, our approach of in-network AIMD anchoring accrues bandwidth savings as well by pushing video chunks from R into the network for caching at a mirror site that can potentially be used by a large number of clients. Here, the location of flow ingress points relative to R determines the extent of bandwidth savings accrued. Given that $\eta_c \gg \eta_s$, the bandwidth savings accrued by our approach of in-network anchored AIMD can be substantial in most cases, relative to the existing approaches of end-point anchored AIMD.

When download paths are not congested, the savings factor is: $\frac{k_h}{K}(1 - \frac{\eta_s}{\eta_c})$, where k_h and K are the number of hops from R to the ingress node and to the client sites respectively. Deeper is the location of ingress point in the network, larger is k_h — and hence more bandwidth savings. When download paths are congested, the savings factor becomes smaller at: $\frac{k_h}{K}(1 - \frac{\eta_s \cdot d_s}{\eta_c \cdot d'_s})$. Referring to Figure 7, the ingress nodes 3 and 7 are 1-hop away each from R — i.e., $k_h = 1$. Consider the case of clients being 3 hops away from R , i.e., $K = 3$. For $\eta_s = 5$ and $\eta_c = 100$, the bandwidth savings in a case of no congestion is 0.32, i.e., 32% savings. We determine the bandwidth savings in a case of congestion, for the data sizes: $d_s = 90$ mbytes and $d'_s = 40$ mbytes. The savings factor is: 0.296 (i.e., 29.6% savings). In some rare scenarios of $\eta_s d_s > \eta_c d'_s$, our in-network AIMD anchoring may even become more expensive. This case can arise for $\eta_s > 44$.

When there is no congestion along the distribution paths, the AIMD anchors manifest as simple and transparent way-points for the video data flows. If there is congestion however, the AIMD anchors purport to adjust the video send rates to clients so that an acceptable QoS can be sustained. Here, the QoS is captured by two attributes: video playback latency and download bit-rate⁷.

4.4 Simulation results on playback latency

For the distribution paths shown in Figure 7, the playback latency for various download rates η_c and chunk push rates η_s is shown in Figure 8 from simulation experiments (results are expressed as normalized values). The parameter ranges $\eta_c \in 0.03 - 0.04$ and $\eta_s \in 0.01 - 0.1$ are of direct interest. Two cases of bandwidth and storage capacities are shown (I and II), with case-II showing more bandwidth-constrained links and nodes when compared to case-I. Capacity allocations in case-II are 25% of that in case-I, with a consequent higher playback latency. For $[\eta_c = 0.035, \eta_s = 0.1]$, the latency in case-II is 22 and case-I is 13. For $\eta_s = 0.2$, the latency in these cases are 26 and 17; for $\eta_s = 0.05$, the latency values are 15 and 8; for $\eta_s = 0.5$, the latency values are 45 and 38. Consider, say, the parameter $\eta_s = 0.1$. An AIMD-computed reduction in η_c from 0.035 to 0.025 lowers the playback latency to 16 in case-II and to 6 in case-I. If η_c is reduced to 0.02, the latency becomes lower at 11 and 4. The QoS improvement due to a latency reduction is

⁷Video playback latency is the time elapsed from a content request of client c to the start of content play-out at c .

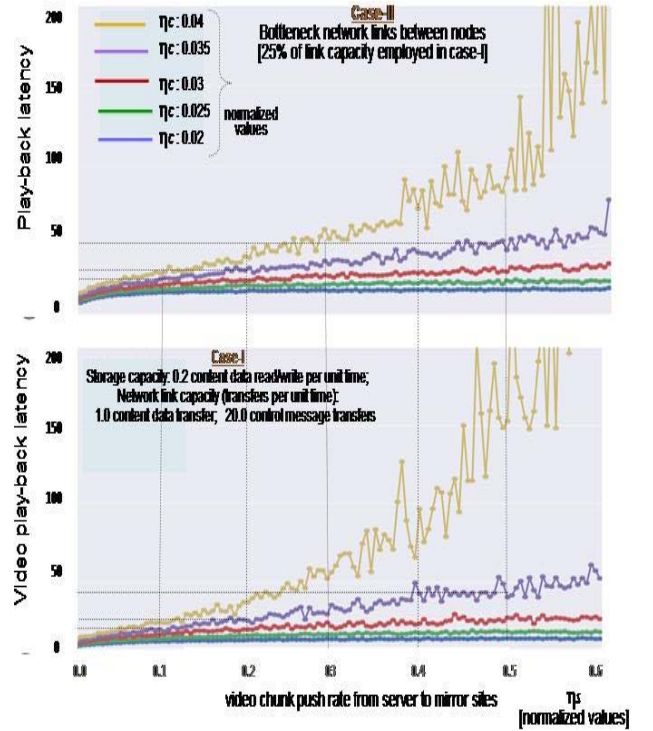


Figure 8: Results on playback latency vs client pull rates

however weighed against the QoS degradation due to any reduction in λ'_c (i.e., effective download bit-rate of video) orchestrated by the AIMD⁸. Note: The effect of AIMD-initiated adjustment of λ'_c on the playback latency is itself not captured by our simulation study shown in Figure 8.

The QoS adaptation process involving an adjustment of video download bit-rate λ'_c (as triggered by AIMD or some other method) falls within the ambit of content delivery functionality of a CDN.

4.5 Selection of AIMD ingress points

The content-caching nodes of a CDN (i.e., mirror sites) are the potential anchors for AIMD ingress points (or, for that matter, any rate controller). One of the video chunks stored at a caching node is selected by the rate controller for transfer over the downstream path towards a client. If needed, a chunk is suitably encoded to lower the transfer bit-rate (by video compression and/or lower frame rate). The ingress node receives the video chunks by a push action from the content server along an upstream path.

⁸Our latency study captures the queuing effects of multiple data flows sharing the storage nodes and network links, but does not factor the processing delays incurred for video encoding & decoding at the ingress & egress nodes. Our prior work [14] gives a detailed analysis of the content delivery latency and overhead from a CDN perspective.

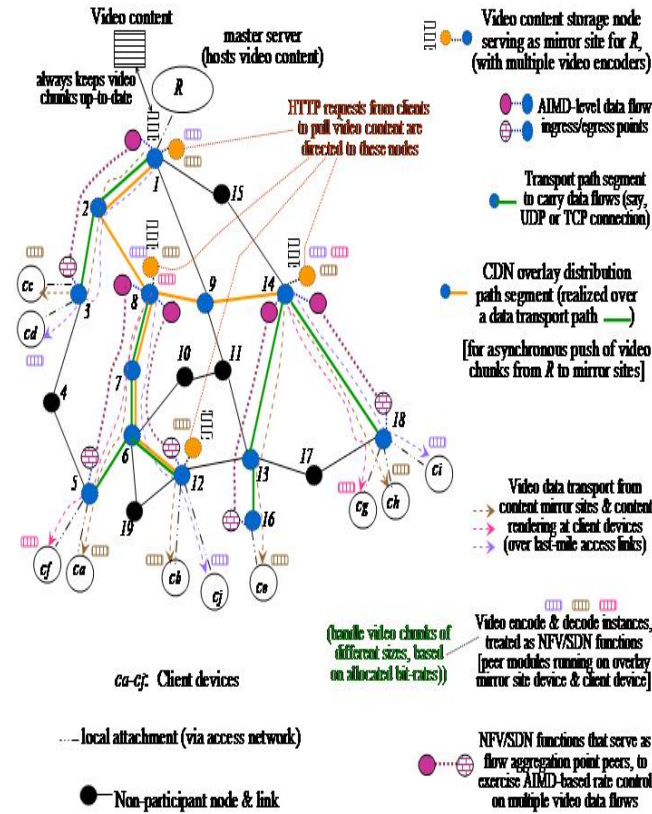


Figure 9: Sample deployment scenario: BAVT-NFV

When the content push paths are disjoint from the content download paths — such as $lnk(0, 3)$ and $lnk(0, 7)$ in Figure 7, resources allocations of these paths can be dimensioned independently. When there is an overlap in one or more these path segments, the likelihood of congestion in the download paths (and hence AIMD-based rate reductions) increases due to bandwidth diversion incurred by the push activities. Alternately, a bandwidth provisioning in the overlapping path segments should factor the combined effects of push and pull activities on the client-level QoS (i.e., video playback latency and download bandwidth).

Consider an alternate distribution tree shown in Figure 9. Content push and download paths overlap in two different segments: $lnk(1, 2)$ and $[lnk(8, 7), lnk(7, 6), lnk(6, 12)]$. The path overlaps manifest as resource-level interactions between the push and pull activities, thereby impacting the QoS. For instance, an increase of η_s may cause a reduction in the download bit-rate λ'_c — and hence lower the QoS. Furthermore, the choice of a node X to serve as mirror site is based on the capability of X to encode video chunks at different rates as demanded by the clients connected to X . For instance, the availability of special-purpose hardware (say, H.264 rack-mount HD video encoding) may be a criteria for X to function as an AIMD ingress point. The placement of mirror sites in the distribution tree may thus factor in their potential use as AIMD

ingress points for the various client clusters sharing the content distribution tree with diverse traffic flow requirements (say, machine-learning techniques to guide the placement — as suggested in [15]).

5 CONCLUSIONS

The paper described the virtualized control functions (NFV) for an end-to-end bandwidth-adaptive video transport system. The NFV realize the well-known AIMD-based video rate adaptation algorithm to handle congestion along the transport path. AIMD is exercised on the aggregated data flows over the path, with a partitioning of control functions at the source ingress and receiver egress points. With fine-granular video encoders in devices, the handling of 'last-mile' issues pertaining to bandwidth sharing in the access network paths was discussed. Specifically, how the propensity of client devices to seek a higher bandwidth share (benign or malicious, conservative or aggressive) can be managed in a collaborative manner was discussed.

We presented experimental results to demonstrate the viability of our NFV architecture. Two levels of NFV were addressed: i) AIMD-level for aggregated video traffic control, and ii) device-level video encoder for per-flow bit-rate control. A deployment scenario for (i) and (ii) in a wide-area Content Distribution Network (CDN) was also discussed.

REFERENCES

- [1] P. Georgopoulos, Y. Elkhatib, M. Broadbent, M. Mu, and N. Race. Towards Network-wide QoE Fairness using OpenFlow-assisted Adaptive Video Streaming. In *proc. ACM-SIGCOMM workshop on Future Human-Centric Multimedia Networking*, Aug. 2013.
- [2] S. Keshav. A Control-Theoretic Approach to Flow Control. In *proc. ACM SIGCOMM*, 1991.
- [3] R. Prasad, C. Dovrolis, M. Murray, and K. Claffy. Bandwidth Estimation: Metrics, Measurement Techniques, and Tools. In *IEEE Network*, 2003.
- [4] C. Patridge. Traffic Shaping. In *Chap. 6, Gigabit Networking*, Addison-Wesley Publ., 1994.
- [5] R. Kuschig, I. Kofler, and H. Hellwagner. An Evaluation of TCP-based Rate-Control Algorithms for Adaptive Internet Streaming of H.264/SVC. In *proc. conf on Multimedia Systems (ACM-SIGMM)*, 2010.
- [6] K. Lee, T. Kim, and V. Bharghavan. A Comparison of End-to-End Congestion Control Algorithms: The case of AIMD and AIPD. In *proc. IEEE INFOCOM*, 2001.
- [7] J. C. Bolot. Characterizing End-to-End Packet Delay and Loss in the Internet. In *proc. ACM SIGCOMM*, San Francisco (CA), 1993.
- [8] K. Ravindran, A. Adithan, and S. Mukhopadhyay. Stability of Video Rate Control Algorithms Over Bandwidth-limited Network Paths. In *proc. IEEE-ACM Conf. on Network Services and Management (CNSM)*, Rio De Janeiro (Brazil), Nov. 2014.
- [9] G. Tian and Y. Liu. Towards Agile and Smooth Video Adaptation in Dynamic HTTP Streaming. In *proc. CoNEXT (ACM-SIGCOMM)*, Dec. 2012.
- [10] K. Ravindran. Software cybernetics to manage adaptation behavior of complex network systems. In *proc. IEEE Intl. Conf. on Computer Communications and Networks (ICCN)*, Shanghai (China), Aug. 2014.
- [11] S. T. Wiegand, H. Schwarz, A. Joch, F. Kossentini, G. Sullivan. Rate-Constrained Coder Control and Comparison of Video Coding Standards. *IEEE Trans. Circ. & Sys. for Video Tech.*, July 2003.
- [12] Pixel Tools. Dynamically adjust encoder parameters to achieve a target bit rate. www.pixeltools.com/rate_control_paper.html, 2012.
- [13] S. Lederer, C. Muller, and C. Timmerer. Dynamic Adaptive Streaming over HTTP Dataset. In *proc. ACM conf. on Multimedia Systems*, Febr. 2012.
- [14] K. Ravindran, Y. Wardei, and M. Iannelli. Model-based Assessment of QoS Adaptation in Complex Networked Systems. In *proc. IEEE Intl. Conf. on System of Systems Engineering (SoSE)*, June 2017.
- [15] W. Hoiles, O. N. Gharehshiran, V. Krishnamurthy, N.D. Dao, and H. Zhang. Adaptive Caching in the YouTube Content Distribution Network: A Revealed Preference Game-Theoretic Learning Approach. In *IEEE Trans. on Cognitive Communications and Networking*, 1(1), Mar. 2015.